

# Closures and Event Loop



tell me about Closures & the event loop in javascript in detail with example

Absolutely. **Closures and the Event Loop** are two of the most important JavaScript concepts once you move beyond basic syntax. They explain a lot of what happens behind the scenes in real-world JavaScript, especially in **React, Node.js, APIs, timers, event handlers, and asynchronous code**.

## 1. Closures in JavaScript

### What is a Closure?

A **closure** happens when an inner function remembers and can access variables from its outer function **even after the outer function has finished executing**.

The easiest way to remember it:

**Closure = Function + its remembered surrounding environment**

### Basic example

JavaScript

```
function outer() {  
  let message = "Hello";  
  
  function inner() {  
    console.log(message);  
  }  
  
  return inner;  
}  
  
const fn = outer();  
  
fn();
```

Output:

Hello

At first, this may look strange.

**outer()** has already finished:

JavaScript

```
const fn = outer();
```

So normally, you might think **message** should disappear.

But it doesn't.

The returned **inner** function **remembers** the **message** variable.

That's a closure.

---

## 2. How the Closure Works

Consider:

```
JavaScript
function outer() {
  let x = 10;

  function inner() {
    console.log(x);
  }

  return inner;
}

const myFunction = outer();
myFunction();
```

Execution happens roughly like this:

### Step 1

JavaScript calls:

```
JavaScript
outer();
```

An execution context is created.

Inside it:

```
JavaScript
x = 10
inner = function...
```

### Step 2

**outer()** returns:

```
JavaScript
inner
```

### Step 3

Normally, you might expect **x** to be gone because **outer()** has finished.

But **inner()** still references **x**.

Therefore JavaScript keeps the required lexical environment alive.

## Step 4

When you execute:

```
JavaScript
myFunction();
```

JavaScript finds:

```
JavaScript
x
```

from the environment where **inner** was created.

## 3. Lexical Scope

Closures depend heavily on **lexical scope**.

Lexical scope means:

A function can access variables based on where the function was written, not where it is called.

Example:

```
JavaScript
let name = "Navi";

function outer() {
  let age = 22;

  function inner() {
    console.log(name);
    console.log(age);
  }

  inner();
}

outer();
```

**inner()** can access:

```
name
age
```

because of its lexical environment.

## 4. Closure Example with Counter

One of the most common practical uses of closures is creating private variables.

```
JavaScript
function createCounter() {
```

```
let count = 0;

return function () {
  count++;
  console.log(count);
};

const counter = createCounter();

counter();
counter();
counter();
```

Output:

```
1
2
3
```

Why?

The returned function remembers:

```
JavaScript
count
```

So even though `createCounter()` finished, `count` remains accessible.

## 5. Private Variables Using Closures

JavaScript doesn't traditionally have private variables using normal variables.

Closures can provide similar behavior.

```
JavaScript
function bankAccount(initialBalance) {
  let balance = initialBalance;

  return {
    deposit(amount) {
      balance += amount;
    },

    withdraw(amount) {
      balance -= amount;
    },

    getBalance() {
      return balance;
    }
  };
}

const account = bankAccount(1000);

account.deposit(500);

console.log(account.getBalance());
```

Output:

```
1500
```

But:

**JavaScript**

```
console.log(account.balance);
```

gives:

```
undefined
```

Because **balance** is private.

Only the returned functions can access it.

## 6. Each Closure Has Its Own Environment

Consider:

**JavaScript**

```
function createCounter() {
  let count = 0;

  return function () {
    count++;
    return count;
  };
}

const counter1 = createCounter();
const counter2 = createCounter();

console.log(counter1());
console.log(counter1());

console.log(counter2());
console.log(counter2());
```

Output:

```
1
2
1
2
```

Why doesn't **counter2** start at 3?

Because each call creates a **separate lexical environment**.

Conceptually:

```
counter1
└─ count = 0

counter2
└─ count = 0
```

They don't share the same `count`.

## 7. Closures with Parameters

Closures can remember function parameters too.

```
JavaScript
function greeting(name) {
  return function () {
    console.log("Hello " + name);
  };
}

const greetNavi = greeting("Navi");

greetNavi();
```

Output:

```
Hello Navi
```

The inner function remembers:

```
JavaScript
name = "Navi"
```

## 8. Function Factory

Closures are useful for creating customized functions.

```
JavaScript
function multiplier(x) {
  return function (number) {
    return number * x;
  };
}

const double = multiplier(2);
const triple = multiplier(3);

console.log(double(10));
console.log(triple(10));
```

Output:

```
20
30
```

You can think of it like:

```
multiplier(2)
  ↓
function remembers x = 2

multiplier(3)
  ↓
function remembers x = 3
```

## 9. Closures with setTimeout

Closures become especially important with asynchronous JavaScript.

```
JavaScript
function createMessage() {
  let message = "Hello";

  setTimeout(function () {
    console.log(message);
  }, 2000);
}

createMessage();
```

After 2 seconds:

```
Hello
```

The outer function has already completed, but the callback remembers **message**.

That's a closure.

## 10. Closure with Loops

This is a very common interview question.

### Using var

```
JavaScript
for (var i = 1; i <= 3; i++) {
  setTimeout(function () {
    console.log(i);
  }, 1000);
}
```

Output:

```
4
4
4
```

Why?

**var** is function-scoped, so all callbacks refer to the same **i**.

By the time the callbacks execute, the loop has finished and:

```
JavaScript
i = 4
```

So all callbacks print 4.

# Using `let`

JavaScript

```
for (let i = 1; i <= 3; i++) {
  setTimeout(function () {
    console.log(i);
  }, 1000);
}
```

Output:

```
1
2
3
```

`let` creates a separate binding for each iteration.

Conceptually:

```
Iteration 1 → i = 1
Iteration 2 → i = 2
Iteration 3 → i = 3
```

Each callback remembers its corresponding value.

## 11. Why Closures Are Important

You'll encounter closures in:

- Event handlers
- `setTimeout`
- `setInterval`
- Promises
- Async functions
- React components
- React hooks
- Callbacks
- Factory functions
- Data privacy
- Memoization
- Currying
- Modules

For example, React code heavily relies on functions retaining access to values from the render in which they were created.

## 12. The Event Loop

Now let's move to the **Event Loop**.

JavaScript is often described as:

**Single-threaded but asynchronous.**

This means JavaScript executes your JavaScript code using a single main execution thread, but the surrounding runtime provides mechanisms for handling asynchronous operations.

For example:

```
JavaScript
console.log("Start");

setTimeout(() => {
  console.log("Timer");
}, 2000);

console.log("End");
```

Output:

```
Start
End
Timer
```

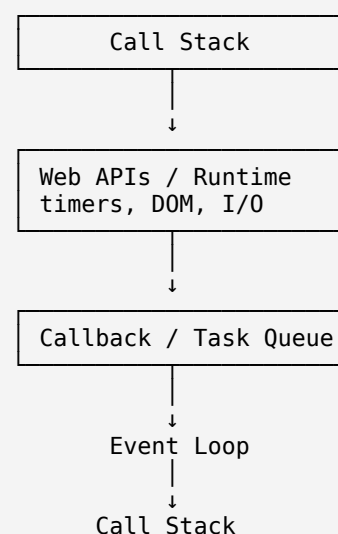
But why?

Because `setTimeout()` doesn't block JavaScript for two seconds.

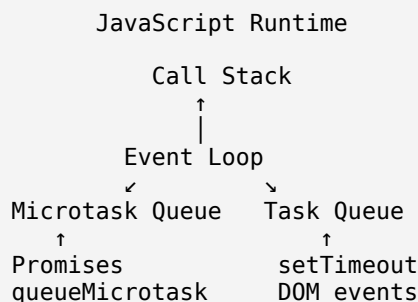
This is where the **Event Loop** comes in.

## 13. The Main Parts

A simplified model contains:



There is also a **microtask queue**, which is extremely important:



## 14. Call Stack

The Call Stack keeps track of currently executing functions.

Example:

```

JavaScript
function one() {
  two();
}

function two() {
  console.log("Hello");
}

one();
  
```

The stack changes approximately like:

```
one()
```

Then:

```
two()
one()
```

Then:

```
console.log()
two()
one()
```

Then everything finishes:

```
empty
```

JavaScript executes functions from the top of the stack.

## 15. What Happens with setTimeout?

Consider:

```

JavaScript
console.log("A");
  
```

```
setTimeout(() => {
  console.log("B");
}, 0);

console.log("C");
```

Output:

```
A
C
B
```

Even with:

```
JavaScript
setTimeout(..., 0)
```

**B** doesn't execute immediately.

Why?

## Step 1

```
JavaScript
console.log("A");
```

Call stack executes it.

Output:

```
A
```

## Step 2

JavaScript encounters:

```
JavaScript
setTimeout(...)
```

The timer is registered with the runtime.

The callback waits until it is eligible to be queued.

JavaScript continues.

## Step 3

```
JavaScript
console.log("C");
```

Output:

```
C
```

## Step 4

The timer callback becomes a task.

The Event Loop eventually moves it onto the Call Stack.

Then:

```
JavaScript
console.log("B");
```

Output:

```
B
```

Final:

```
A
C
B
```

## 16. The Microtask Queue

Now we get to one of the most important parts.

Promises use the **microtask queue**.

Example:

```
JavaScript
console.log("Start");

Promise.resolve().then(() => {
  console.log("Promise");
});

console.log("End");
```

Output:

```
Start
End
Promise
```

The Promise callback doesn't execute immediately.

It goes into the **microtask queue**.

## 17. Microtasks vs Tasks

Consider:

```
JavaScript
console.log("1");

setTimeout(() => {
```

```

    console.log("2");
  }, 0);

  Promise.resolve().then(() => {
    console.log("3");
  });

  console.log("4");

```

Output:

```

1
4
3
2

```

Why?

Execution:

## Synchronous code first

```

JavaScript
console.log("1");

```

→ 1

Then:

```

JavaScript
setTimeout(...)

```

Timer is registered.

Then:

```

JavaScript
Promise.resolve().then(...)

```

Promise callback enters the microtask queue.

Then:

```

JavaScript
console.log("4");

```

→ 4

Now the Call Stack is empty.

The Event Loop checks microtasks.

```

Microtask Queue:
Promise callback

```

So:

3

Then it processes the task queue:

`setTimeout callback`

So:

2

Final:

1  
4  
3  
2

## 18. Important Priority

A simplified priority model is:

```
1. Synchronous JavaScript
   ↓
2. Microtasks
   - Promise.then()
   - Promise.catch()
   - Promise.finally()
   - queueMicrotask()
   ↓
3. Tasks
   - setTimeout
   - setInterval
   - many DOM events / runtime callbacks
```

So generally:

**Microtasks are processed before the next task.**

## 19. A Very Important Example

Look at this:

```
JavaScript
console.log("Start");

setTimeout(() => {
  console.log("Timeout");
}, 0);

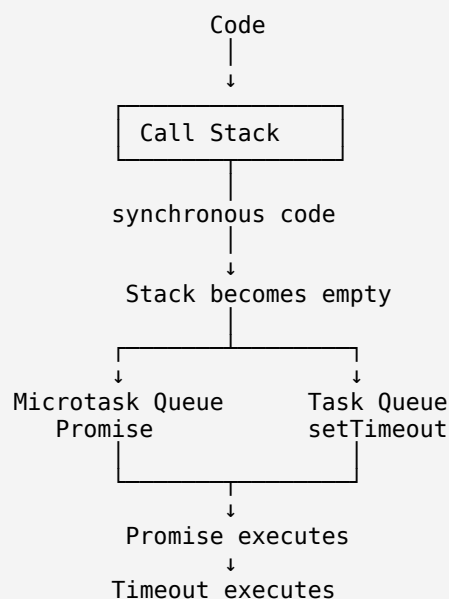
Promise.resolve().then(() => {
  console.log("Promise");
});

console.log("End");
```

Execution:

Start  
End  
Promise  
Timeout

Think:



## 20. Event Loop + Closures Together

This is where the two concepts become really interesting.

Consider:

```

JavaScript
function createMessage() {
  let message = "Hello from closure";

  setTimeout(() => {
    console.log(message);
  }, 1000);
}

createMessage();
  
```

There are two concepts here:

### Closure

The callback:

```

JavaScript
() => {
  console.log(message);
}
  
```

remembers:

```

JavaScript
  
```

*message*

## Event Loop

The callback doesn't execute immediately.

The timer schedules it to execute later.

So:

```

createMessage()
  ↓
message created
  ↓
setTimeout registers callback
  ↓
createMessage finishes
  ↓
closure keeps message accessible
  ↓
1 second passes
  ↓
callback enters task queue
  ↓
Event Loop
  ↓
Call Stack
  ↓
console.log(message)

```

That's a very important relationship to understand.

## 21. Async/Await and Event Loop

Since you've been learning Promises and **async/await**, this is especially important.

Consider:

```

JavaScript
async function example() {
  console.log("A");

  await Promise.resolve();

  console.log("B");
}

console.log("C");

example();

console.log("D");

```

Output:

```

C
A
D
B

```

Why?

Initially:

```
console.log("C")
```

→ **C**

Then:

```
JavaScript
example();
```

Inside **example()**:

```
A
```

is printed.

Then:

```
JavaScript
await Promise.resolve();
```

The async function pauses at **await**.

The continuation:

```
JavaScript
console.log("B");
```

is scheduled as a microtask.

Then JavaScript continues:

```
JavaScript
console.log("D");
```

→ **D**

Then microtasks execute:

```
B
```

Final:

```
C
A
D
B
```

## 22. Event Loop Interview Question

Consider:

### JavaScript

```
console.log("1");

setTimeout(() => {
  console.log("2");
}, 0);

Promise.resolve().then(() => {
  console.log("3");
});

Promise.resolve().then(() => {
  console.log("4");
});

console.log("5");
```

Answer:

```
1
5
3
4
2
```

Reason:

### Synchronous:

```
1
5
```

### Microtasks:

```
3
4
```

### Task:

```
2
```

## 23. Nested Microtasks

Here's another interesting example:

### JavaScript

```
console.log("A");

Promise.resolve().then(() => {
  console.log("B");

  Promise.resolve().then(() => {
    console.log("C");
  });
});

console.log("D");
```

Output:

```
A
D
B
C
```

Why?

Initially:

```
A
D
```

Then first Promise executes:

```
B
```

While executing it, another Promise is added to the microtask queue.

That microtask is then processed:

```
C
```

## 24. Nested setTimeout

Consider:

```
JavaScript
console.log("A");

setTimeout(() => {
  console.log("B");

  setTimeout(() => {
    console.log("C");
  }, 0);
}, 0);

console.log("D");
```

Output:

```
A
D
B
C
```

The first timer runs as a task.

Inside that callback, another timer is registered.

That second timer must wait for a later task turn.

## 25. Closures + Event Handlers

Closures are extremely common in browser development.

### JavaScript

```
function setupButton() {
  let clicks = 0;

  const button = document.querySelector("#btn");

  button.addEventListener("click", () => {
    clicks++;

    console.log(`Clicked ${clicks} times`);
  });
}

setupButton();
```

The `clicks` variable belongs to `setupButton()`.

But the click handler remembers it.

Every time the user clicks:

```
click 1 → clicks = 1
click 2 → clicks = 2
click 3 → clicks = 3
```

That's a closure.

The browser event system triggers the callback later, while the callback retains access to its lexical environment.

## 26. Common Misunderstanding

A very important point:

### ✗ Closure is not:

"A function that executes another function."

### ✗ Closure is not:

"Any function inside another function."

A nested function can form a closure when it **retains access to variables from an outer lexical scope**.

For example:

### JavaScript

```
function outer() {
  let x = 10;

  return function inner() {
    return x;
  };
}
```

Here `inner` closes over `x`.

## 27. Closure Memory

Closures can keep variables alive as long as the closure remains reachable.

For example:

```
JavaScript
function createHugeData() {
  const data = new Array(1000000).fill("data");

  return function () {
    console.log(data.length);
  };
}

const fn = createHugeData();
```

The returned function still references **data**.

Therefore the data may remain reachable as long as **fn** is reachable.

This is useful, but it also means **careless closures can contribute to memory usage**.

## 28. A Practical Mental Model

When you see a function, ask:

### Question 1

**What variables can this function access?**

That's lexical scope.

### Question 2

**Does the function survive after its outer function finishes?**

If yes, it may be using a closure.

### Question 3

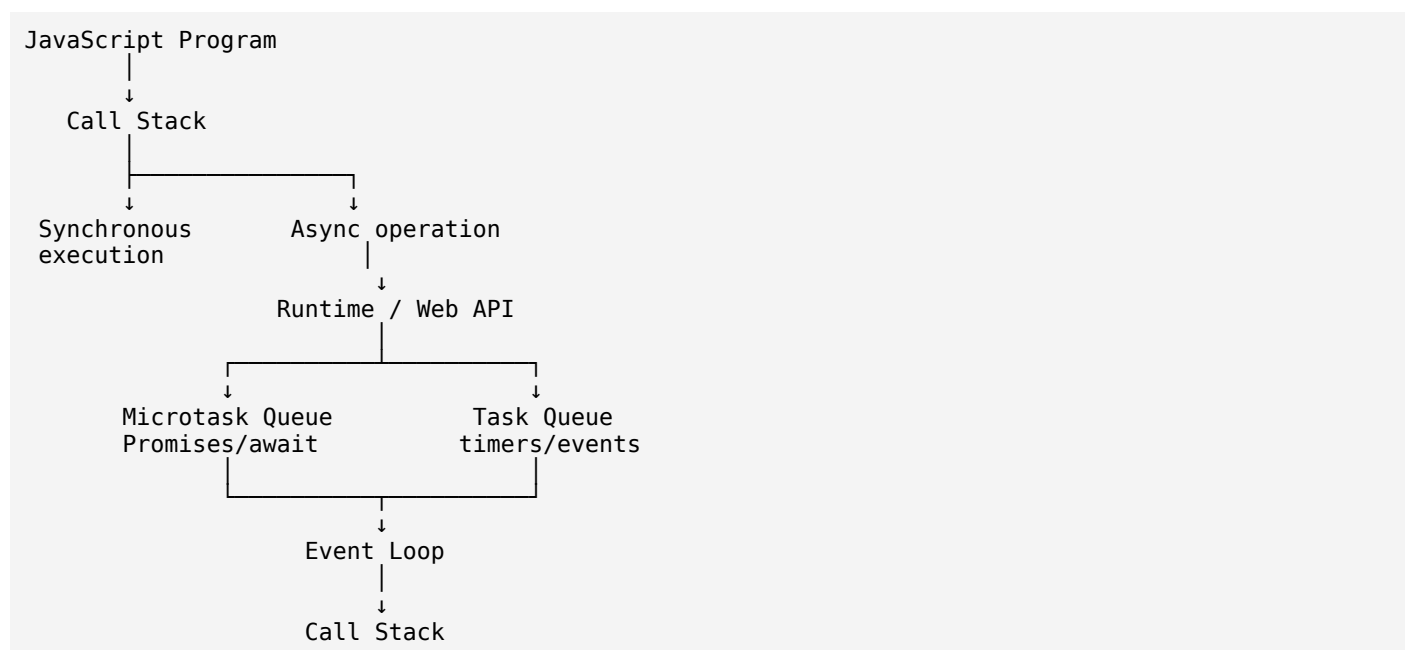
**When will this function execute?**

If asynchronous, think about:

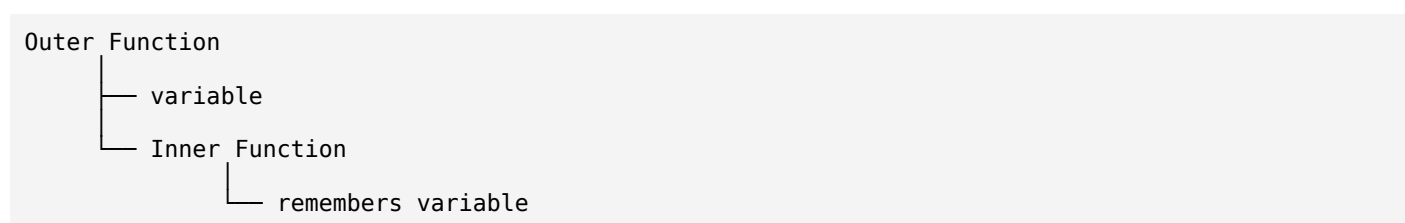
```
Call Stack
↓
Runtime/Web APIs
↓
Microtask/Task Queue
↓
Event Loop
↓
Call Stack
```

## 29. The Big Picture

Put everything together:



And closures are about **what functions remember**:



## 30. What You Should Remember for Interviews

### Closures

#### Definition:

A closure is a function bundled with access to variables from its lexical environment, allowing it to retain those variables even after the outer function has finished executing.

Know these examples:

```

JavaScript
function counter() {
  let count = 0;

  return () => ++count;
}
  
```

```

JavaScript
function multiplier(x) {
  return n => n * x;
}
  
```

```

JavaScript
for (let i = 0; i < 3; i++) {
  // ...
}
  
```

```
    setTimeout(() => console.log(i), 0);
  }
```

## Event Loop

Remember:

JavaScript executes synchronous code first. Asynchronous callbacks are scheduled by the runtime, and the Event Loop coordinates when queued callbacks can return to the Call Stack.

And especially:

```
Synchronous code
  ↓
Microtasks
  ↓
Tasks
```

So:

```
JavaScript
console.log("A");

setTimeout(() => console.log("B"), 0);

Promise.resolve().then(() => console.log("C"));

console.log("D");
```

produces:

```
A
D
C
B
```

## The most important connection

If you understand this example, you've understood a **large part of both topics**:

```
JavaScript
function createCounter() {
  let count = 0;

  return function () {
    setTimeout(() => {
      count++;
      console.log(count);
    }, 1000);
  };
}

const counter = createCounter();

counter();
counter();
```

There are **two concepts working together**:

## Closure:

```
JavaScript  
count
```

is remembered by the callback.

## Event Loop:

```
JavaScript  
setTimeout(...)
```

causes the callback to execute asynchronously later.

So the mental model is:

**Closure tells you what the callback remembers. Event Loop tells you when the callback gets a chance to execute.**

That distinction is extremely useful when learning **Promises, async/await, React, Node.js, and event-driven programming.**

ChatGPT can make mistakes. Check important info.